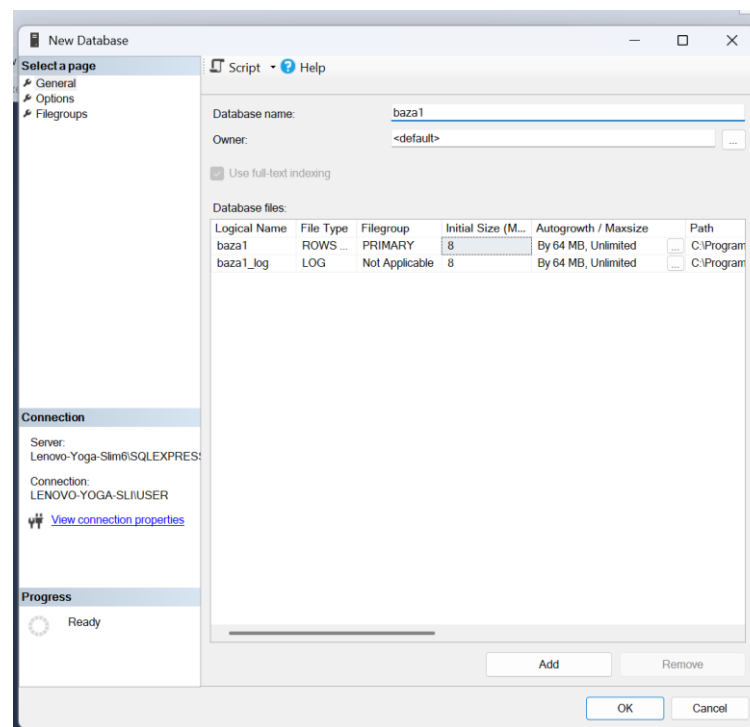
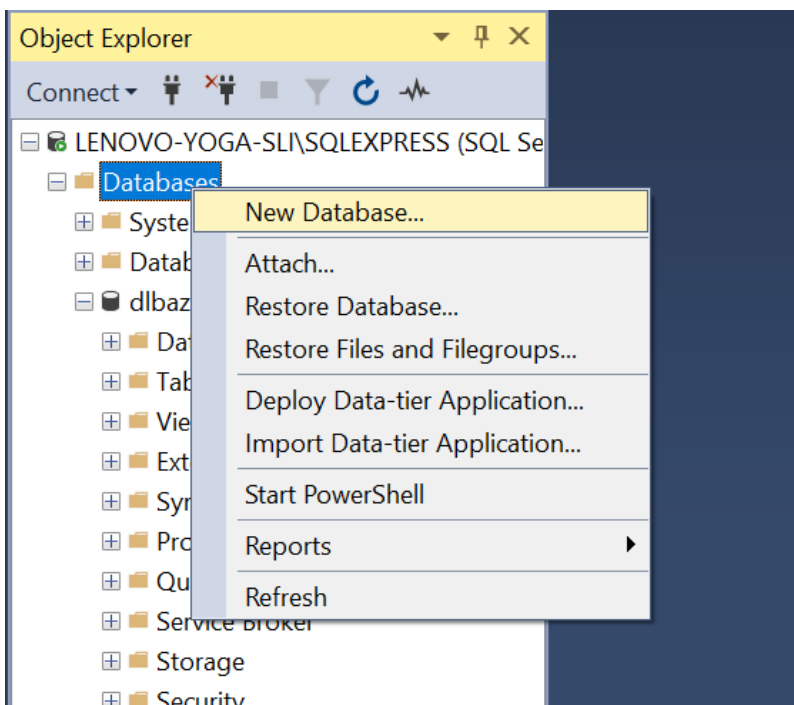


Tworzenie schematów tabel

Ćw. Nr 8

Tworzenie nowej bazy danych

- ▶ Tworzenie nowej bazy danych:



Tworzenie tabel

Sposoby tworzenia tabel w bazie danych:

Za pomocą interfejsu graficznego

Za pomocą polecenia **CREATE TABLE**

Tworzenie tabel - interfejs graficzny

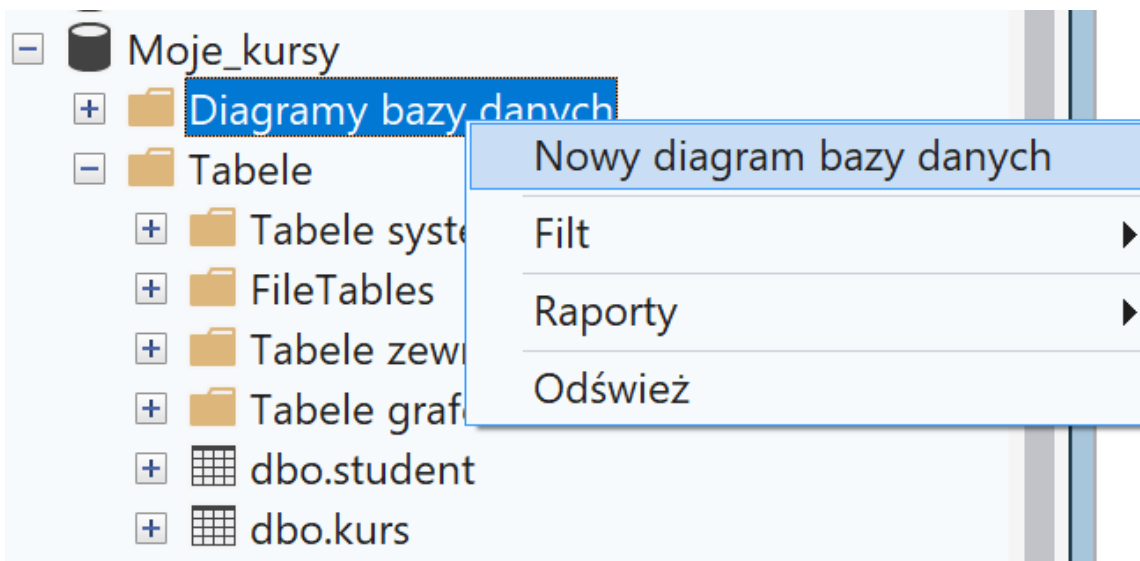
► Tworzenie tabel:

The screenshot illustrates the process of creating a new table in SQL Server. On the left, the 'Object Explorer' pane shows the database hierarchy for 'LENOVO-YOGA-SLI\SQLEXPRESS (SQL Se...)' with 'd1baza' selected. A right-click context menu is open over the 'Tables' folder, with the 'New' option selected, leading to a sub-menu where 'Table...' is highlighted. On the right, a preview window titled 'Lenovo-Yoga-Slim6...baza - dbo.Table_1*' displays the table structure with the following columns:

Column Name	Data Type	Allow Nulls
nrAlbumu	nchar(10)	☐
		☐

Tworzenie diagramu bazy

- ▶ Tworzenie diagramu bazy danych:



Tworzenie tabel - instrukcja

- Tworzenie tabel - instrukcja **CREATE TABLE**

```
CREATE TABLE dbo.Nazwa_tabeli (  
    Nazwa_kolumny typ_danych parametry,  
    ....  
);
```

- Przykład:

```
CREATE TABLE przedmiot (  
    przedmiotID nvarchar(10) PRIMARY KEY,  
    nazwisko nvarchar(20) not null,  
    przedmiot nvarchar(50) not null,  
    typ nvarchar(10) not null  
);
```

Wstawianie danych do tabeli

- Wstawianie danych - instrukcja INSERT INTO

```
INSERT INTO nazwa_tabeli  
VALUES  
(lista_wartości)
```

- Przykład:

```
INSERT INTO przedmioty  
(nazwisko, przedmiot, typ)  
VALUES  
( 'Nowacka', 'Hiszpanski', 'WYK' ),  
( 'Majka', 'Hiszpanski', 'LAB' ),  
( 'Baka', 'Geologia', 'WYK' )
```

Modyfikowanie danych

- Modyfikowanie danych - instrukcja **UPDATE**:
umożliwia zmianę wartości w istniejących rekordach

UPDATE tabela

SET kolumna = nowa_wartość

WHERE warunek

Polecenie UPDATE kończymy klauzulą WHERE, która określa jakie wiersze zostaną zaktualizowane. Brak WHERE - oznacza zmianę wszystkich rekordów.

- Przykład:

```
UPDATE Klienci
```

```
SET kl_mail = 'market@medial.pl'
```

```
WHERE klientID=156
```

Usuwanie danych

- Polecenie **DELETE** usuwa rekordy z tabeli.

```
Delete  
From tabela  
Where warunek
```

Instrukcja DELETE usuwa wybrane lub wszystkie wiersze z tabeli, ale nie usuwa samej tabeli.

- Przykład:

```
DELETE from Klienci  
WHERE klientID = 212
```

Tworzenie i usuwanie indeksów

- Indeksy przyspieszają wyszukiwanie danych w tabelach. Najczęściej używa się ich dla kolumn często występujących w klauzulach WHERE, JOIN, ORDER BY.
- Tworzenie indeksu:
`CREATE INDEX idx_nazwisko`
`ON Pracownicy(Nazwisko);`
- Usuwanie indeksu:
`DROP INDEX idx_nazwisko`
`ON Pracownicy`

Constrainty

Constrainty (ograniczenia) w SQL zapewniają:

- **poprawność danych** - np. pensja nie może być ujemna
- **spójność i integralność danych** przechowywanych w tabelach - np. nie wstawi się zamówienia z nieistniejącym klientem
- **bezpieczeństwo danych** - zapobiegają zapisaniu błędnych, sprzecznych lub niepożądanych danych.
- **właściwą jakość danych** — niezależnie od aplikacji, użytkownika czy interfejsu.
- **automatyczną kontrolę**

Constrainty c.d.

Rodzaje ograniczeń:

1. PRIMARY KEY

- Zapewnia unikalną identyfikację każdego wiersza w tabeli
- Nie może być NULL, musi być unikalny

Przykład:

```
CREATE TABLE Klienci (  
    KlientID INT PRIMARY KEY,  
    Imie VARCHAR(50),  
    Nazwisko VARCHAR(50)  
);
```

Constrainty c.d.

2. FOREIGN KEY

- Utrzymuje **integralność referencyjną** między tabelami. Zapobiega np. usunięciu klienta, jeśli ma jakieś zamówienia.

Przykład:

```
CREATE TABLE Zamowienia (  
    ZamowienieID INT PRIMARY KEY,  
    KlientID INT,  
    FOREIGN KEY (KlientID) REFERENCES Klienci(KlientID)  
);
```

Constrainty c.d.

3. NOT NULL

- Wymusza podanie wartości — kolumna nie może być pusta.

Przykład:

Imie VARCHAR(50) NOT NULL

4. UNIQUE

- Zapewnia unikalność wartości w kolumnie.

Przykład — unikalne adresy e-mail:

Email VARCHAR(100) UNIQUE

Constrainty

5. CHECK

- Wymusza określony warunek logiczny.

Przykład – cena musi być dodatnia:

Cena DECIMAL(10,2) CHECK (Cena > 0)

Przykład – wiek musi być >= 18:

Wiek INT CHECK (Wiek >= 18)

6. DEFAULT

- Ustala wartość domyślną, jeśli nie podano jej w INSERT.

Przykład:

DataRejestracji DATE DEFAULT GETDATE()

Constraints c.d.

7. AUTO_INCREMENT / IDENTITY (w zależności od systemu)

- Automatycznie zwiększa wartość klucza głównego.

Przykład (SQL Server):

```
KlientID INT IDENTITY(1,1) PRIMARY KEY
```

Przykład (MySQL):

```
KlientID INT AUTO_INCREMENT PRIMARY KEY
```

Modyfikowanie tabeli

- Polecenie **ALTER TABLE** - służy do modyfikowania struktury istniejącej tabeli w bazie danych.

ALTER TABLE nazwa tabeli
[rodzaj modyfikacji]

Rodzaje modyfikacji:

ADD | DROP | ADD CONSTRAINT | DROP CONSTRAINT

Modyfikowanie tabeli - przykłady

- **Dodawanie kolumn:**

```
ALTER TABLE nazwa_tabeli
```

```
ADD nazwa_kolumny typ_danych [Ograniczenia]
```

Przykład:

```
ALTER TABLE Produkty
```

```
ADD Cena_Netto DECIMAL(10, 2) NOT NULL;
```

- **Dodawanie constraintów - Klucz Główny**

```
ALTER TABLE nazwa_tabeli
```

```
ADD CONSTRAINT nazwa_PK PRIMARY KEY (kolumna)
```

Przykład:

```
ALTER TABLE Student
```

```
ADD CONSTRAINT PK_Student
```

```
PRIMARY KEY (StudentID);
```

Modyfikowanie tabeli - przykłady

- **Dodawanie constraintów - Klucz obcy**

```
ALTER TABLE nazwa_tabeli  
ADD CONSTRAINT nazwa_FK FOREIGN KEY (kolumna)  
REFERENCES inna_tabela(kolumna);
```

Przykład:

```
ALTER TABLE Loans  
ADD CONSTRAINT FK_Loans_Students  
FOREIGN KEY (StudentID)  
REFERENCES Students(StudentID);
```

- **Dodawanie wartości UNIQUE:**

```
ALTER TABLE nazwa_tabeli  
ADD CONSTRAINT nazwa_unique UNIQUE (kolumna)
```

Zadanie

- ❑ Utwórz bazę danych z następującymi tabelami: Kursy, Studenci, Zapisy.

Kursy:

KursID	Nazwa_Kursu	ECTS
10	Bazy Danych	6
11	Algorytmy	6
12	Sieci Komputerowe	4
13	Programowanie obiektowe	6

Studenci:

StudentID	Imie	Nazwisko	Rok_Studiow
1	Anna	Kowalska	1
2	Jan	Nowak	2
3	Maria	Lewandowska	3
4	Piotr	Zieliński	1
5	Kamil	Zawadzki	2

Zapisy:

ZapisID	StudentID	KursID	Data_zapisu
100	1	10	1.10.2025
101	1	11	1.10.2025
102	2	10	3.10.2025
103	3	12	4.10.2025
104	4	13	4.10.2025

Zadanie c.d.

Wykonaj następujące polecenia:

1. Uzupełnij tabele danymi korzystając z polecenia INSERT INTO. Dobierz typy danych, ustal klucze główne i klucze obce.
2. Zaktualizuj dane:
 - zmień rok studiów wybranego studenta o ID=1 na rok wyższy,
 - zwiększ o 1 liczbę ECTS dla kursu o numerze ID=11.
3. Wyświetl wszystkie dane z tabeli Zapisy.
4. Utwórz indeks na kolumnie Nazwisko w tabeli Studenci.
5. Do tabeli Studenci dodaj pole **email** [*typ danych: varchar(100)*] i uzupełnij danymi:
akowska@student.edu.pl
jnowak@student.edu.pl
pielinski@student.edu.pl
7. Wyświetl dane studentów, którzy nie mają wpisanego adresu email.
8. Usuń studenta, który nie ma żadnego wpisu w tabeli ZAPISY.
9. Zmień typ pola email na **varchar(50)**